Project Title

IntelliSpell: Intelligent Spell Checker using Levenshtein Distance

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

S. Manvi Vaishnavi	(2520030326)
Rohini Tanmayi D	(2520030349)
Pramod Krishna M	(2520030111)

Team Number: 11

Section: 01

Under the Esteemed guidance of

Dr. Ch Anuradha

professor CSE

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DECLARATION

The Mini Project Report entitled "**IntelliSpell: Intelligent Spell Checker using Levenshtein Distance**" is a record of bonafide work of S. Manvi Vaishnavi – 2520030326, Rohini Tanmayi D – 2520030349, Pramod Krishna M – 2520030111, submitted in partial fulfilment for the award of B.Tech in the Department of Computer Science Engineering to K L University, Hyderabad. The results embodied in this report have not been copied from any other department, university or institute.

S. Manvi Vaishnavi – 2520030326

Rohini Tanmayi D – 2520030349

Pramod Krishna M – 2520030111

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

CERTIFICATE

This is to certify that the mini project based report entitled "**IntelliSpell: Intelligent Spell Checker using Levenshtein Distance**" is a bonafide work done and submitted by S. Manvi Vaishnavi – 2520030326, Rohini Tanmayi D – 2520030349, Pramod Krishna M – 2520030111, in partial fulfilment of the requirement for the course Data Structures and Algorithms -3 (25CS2103E), during the academic year 2026-2027.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guide, Dr. Ch Anuradha, Professor CSE, which cannot be expressed in words, for her tremendous support, guidance and encouragement throughout this project.

We express our gratitude to Dr. P.Venkateshwar Rao (CSE)/, Head of the Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

S. Manvi Vaishnavi – 2520030326

Rohini Tanmayi D – 2520030349

Pramod Krishna M – 2520030111

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

IntelliSpell is a browser-based intelligent spell checker built on the Levenshtein edit distance algorithm. It splits text into words, checks each one against a dictionary of 9,844 words, and for every unknown word ranks up to three suggestions within an edit distance of two.

Users can type or paste text, or open a .txt, .pdf or .docx file. Misspelled words are underlined; clicking one opens its suggestions and the chosen word replaces the error right there in the text. The corrected text can be copied or downloaded.

All processing happens locally in the browser, so no document leaves the device. An interactive matrix view shows how each distance is calculated, making the algorithm transparent as well as fast.

Team Name: IntelliSpell

Team Members:

1. S. Manvi Vaishnavi 2520030326
2. Rohini Tanmayi D 2520030349
3. Pramod Krishna M 2520030111

2. INTRODUCTION

Spelling errors such as “recieve”, “seperate” and “definately” are easy to make and easy to miss. **Spell checking** means finding a word that is not valid and suggesting the intended one.

Edit distance measures how different two words are by counting single-letter insertions, deletions and substitutions. **IntelliSpell** applies this idea in the browser: type text or open a file, click a red word and fix it instantly.

Levenshtein distance

The Levenshtein distance between two words is the fewest single-letter edits needed to turn one word into the other. Every edit costs 1:

- Insert a letter = 1
- Delete a letter = 1
- Substitute a letter = 1

For example, “wrod” → “word” needs two substitutions, so its distance is 2. The distance is computed with the Wagner–Fischer dynamic-programming matrix, where each cell holds the cheapest way to turn one prefix into another.

3. LITERATURE SURVEY

The following earlier work shaped the design of IntelliSpell:

Work	Contribution	Use in IntelliSpell
Damerau (1964)	Studied common spelling mistakes such as adding, removing, changing or swapping letters.	Helps us handle common typing mistakes, especially swapped letters like “teh” → “the”.
Levenshtein (1965/66)	Introduced measuring how many insertions, deletions and substitutions are needed to change one word into another.	We use Levenshtein distance to find and rank similar words.
Burkhard & Keller (1973)	Proposed the BK-tree, which helps search for words close to a given word using a distance measure.	Can be used later to make IntelliSpell faster when the dictionary is very large.
Wagner & Fischer (1974)	Developed a dynamic-programming method for finding the edit distance between two strings.	Our project uses the matrix / dynamic-programming approach to calculate word similarity.
Kukich (1992)	Reviewed different methods used for automatically detecting and correcting spelling errors.	Helps us understand how dictionary-based and edit-distance methods are used in spell checkers.
Norvig (2007)	Demonstrated a simple spelling corrector that generates possible words within a small edit distance.	Our project follows a similar idea: dictionary + small edit distance → possible corrections.

4. Problem Statement and Objectives

Problem Statement:

Typing mistakes are easy to make and easy to miss, and most checkers either send private text to a server or give no explanation for their suggestions.

Problem Description:

A useful checker must spot the wrong word, measure how far it is from real words, and suggest the right one, quickly and without sending private text to a server. It should be accurate (find the intended word, not just any word), fast (feel instant, even on long text), private (documents never leave the device) and flexible (typed text and common file formats).

Objectives:

1. Detect misspelled words by checking every word against a dictionary.
2. Measure similarity between words using Levenshtein distance.
3. Rank the closest suggestions: up to three, within distance 2.
4. Let the user correct a word in place with one click.
5. Support typed text and .txt, .pdf and .docx files, and export the result.
6. Keep all processing in the browser to protect privacy.
7. Make the algorithm visible with an interactive matrix.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Any laptop, desktop or phone
- Minimum 4 GB RAM, dual-core processor
- No GPU needed
- No server or database needed
- Internet only for first PDF/DOCX use (reader libraries load from a CDN)

Software Requirements:

- HTML5, CSS3 and JavaScript (no framework, no backend)
- pdf.js 3.11.174 for .pdf files, mammoth 1.6.0 for .docx files
- IBM Plex Serif, Sans and Mono fonts
- Browser: Chrome, Edge or Firefox
- VS Code with Live Server, or Python http.server, to run it locally

6. IMPLEMENTATION

The project is organised into four modules. The flow of a check is: Input → Tokenize → Dictionary lookup → (if unknown) Length filter → Levenshtein (distance ≤ 2 kept) → Rank (closest first, top 3) → Underline + popup → Replace in place, then copy or download.

Module 1: Levenshtein distance (dynamic programming)

Builds the (m+1) × (n+1) matrix row by row. Each cell is the minimum of a deletion, an insertion or a substitution. Time O(m·n); only two rows are kept, so memory is O(n).

```
function levenshtein(a, b) {
  let prev = Array.from({ length: b.length + 1 }, (_, j) => j);
  for (let i = 1; i <= a.length; i++) {
    const cur = [i];
    for (let j = 1; j <= b.length; j++) {
      const cost = a[i - 1] === b[j - 1] ? 0 : 1;
      cur[j] = Math.min(prev[j] + 1,        // delete
                        cur[j - 1] + 1,     // insert
                        prev[j - 1] + cost); // substitute
    }
    prev = cur;
  }
  return prev[b.length];
}
```

Module 2: Dictionary loading and tokenizing

The 9,844 words are stored in a Set for O(1) average lookup and grouped by length so the length filter can skip most of the dictionary. The text is split into words with a regular expression that keeps each word's position, so it can be replaced in place.

```
const DICT = new Set(words);
const BY_LEN = {};
for (const w of words) (BY_LEN[w.length] ||= []).push(w);

function tokenize(text) {
  const out = [], re = /[A-Za-z]+(?:'[A-Za-z]+)?/g;
  let m;
  while ((m = re.exec(text)))
    out.push({ word: m[0], start: m.index, end: m.index + m[0].length });
  return out;
}
```

Module 3: Suggestion ranking

Only words whose length differs by at most 2 are compared. Candidates within distance 2 are kept and sorted closest first; a swap of two neighbouring letters is counted as one edit when ranking, so “teh” finds “the” first. The top three are returned.

```
function suggest(word) {
  const w = word.toLowerCase(), found = [];
  for (let len = w.length - 2; len <= w.length + 2; len++)
    for (const cand of BY_LEN[len] || []) {
      const d = levenshtein(w, cand);
      if (d <= 2) found.push({ cand, d: isSwap(w, cand) ? 1 : d });
    }
  found.sort((x, y) => x.d - y.d);
  return found.slice(0, 3).map(f => f.cand);
}
```

Module 4: Checking, in-place replacement and file input

Every unknown word is underlined. Clicking one opens a popup of suggestions, and the chosen word is spliced into the text at the stored position. Files are read locally: .txt with FileReader, .pdf with pdf.js and .docx with mammoth.

```
function check(text) {
  return tokenize(text)
    .filter(t => !DICT.has(t.word.toLowerCase()))
    .map(t => ({ ...t, suggestions: suggest(t.word) }));
}

function replaceWord(text, err, choice) {
  return text.slice(0, err.start) + choice + text.slice(err.end);
}
```

7. RESULTS & DISCUSSION

IntelliSpell was tested on 20 common typing mistakes. A test passes when the intended word comes back as the first suggestion.

Typed	Top suggestions	Expected	Match
recieve	receive, believe, recipe	receive	Yes – 1st
seperate	separate, operate, generate	separate	Yes – 1st
teh	the, tech, tel	the	Yes – 1st
wrod	word, wood, rod	word	Yes – 1st
freind	friend, find, trend	friend	Yes – 1st
wierd	weird, wired, were	weird	Yes – 1st
neccessary	necessary	necessary	Yes – 1st
enviroment	environment, environments	environment	Yes – 1st

- **20 / 20** – the correct word came first.
- **20 / 20** – the correct word was in the top three.
- **~7 ms** – average time for one misspelled word.

Output screenshots

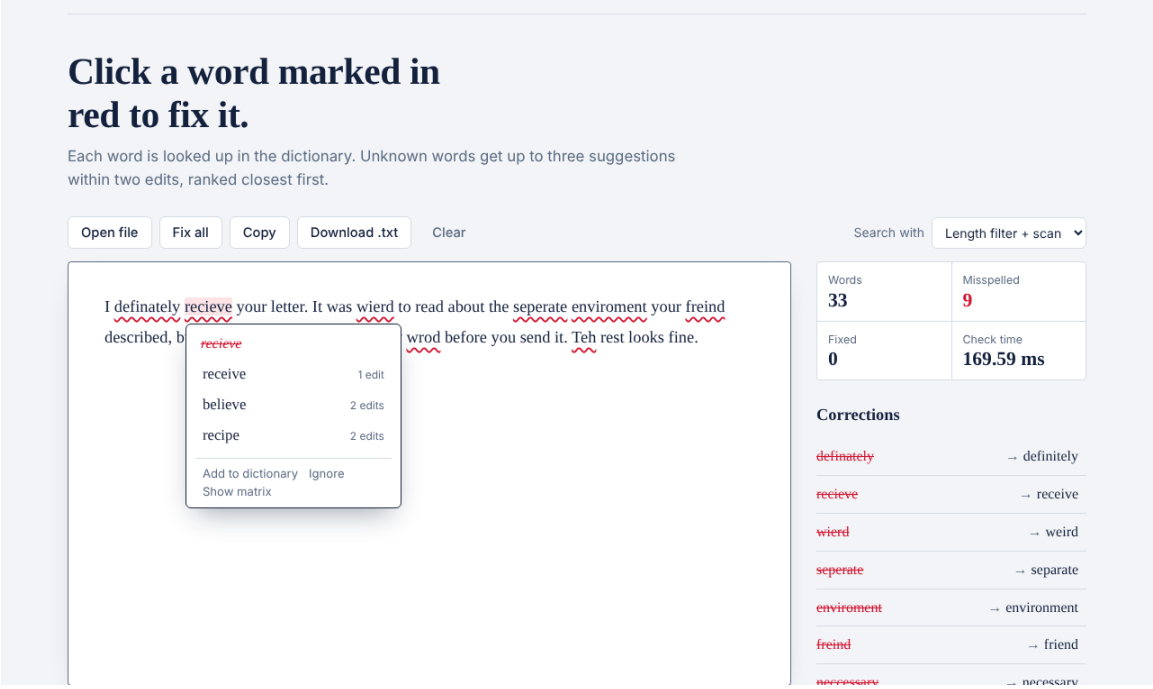


Fig. 1

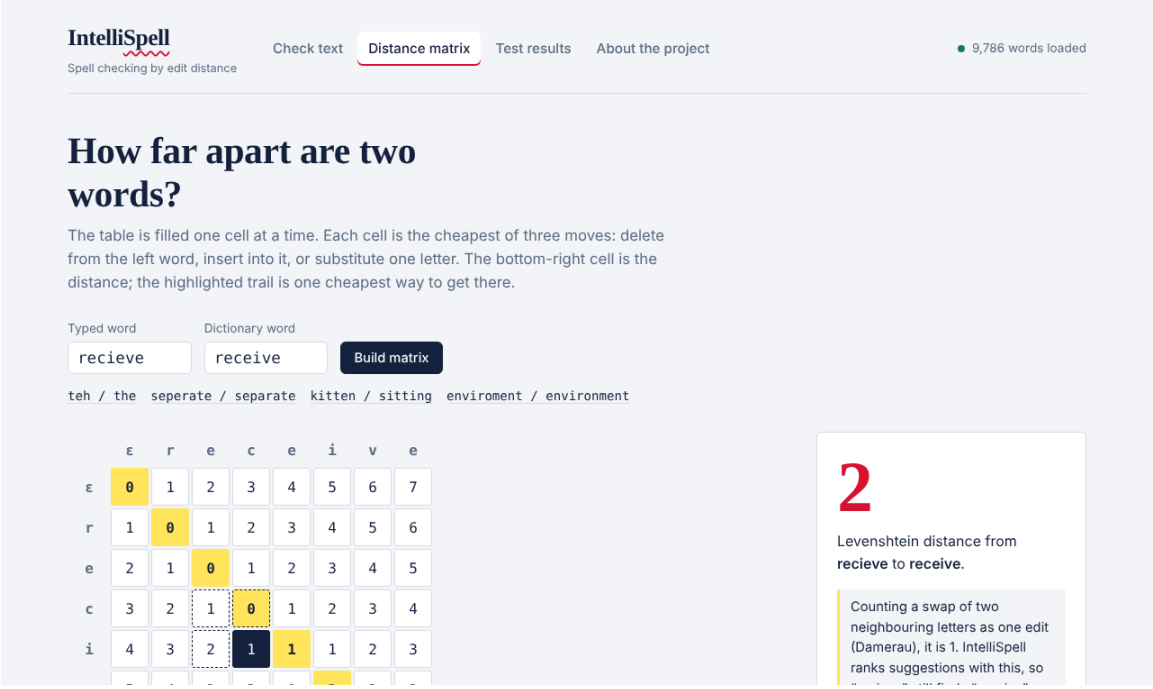


Fig. 2

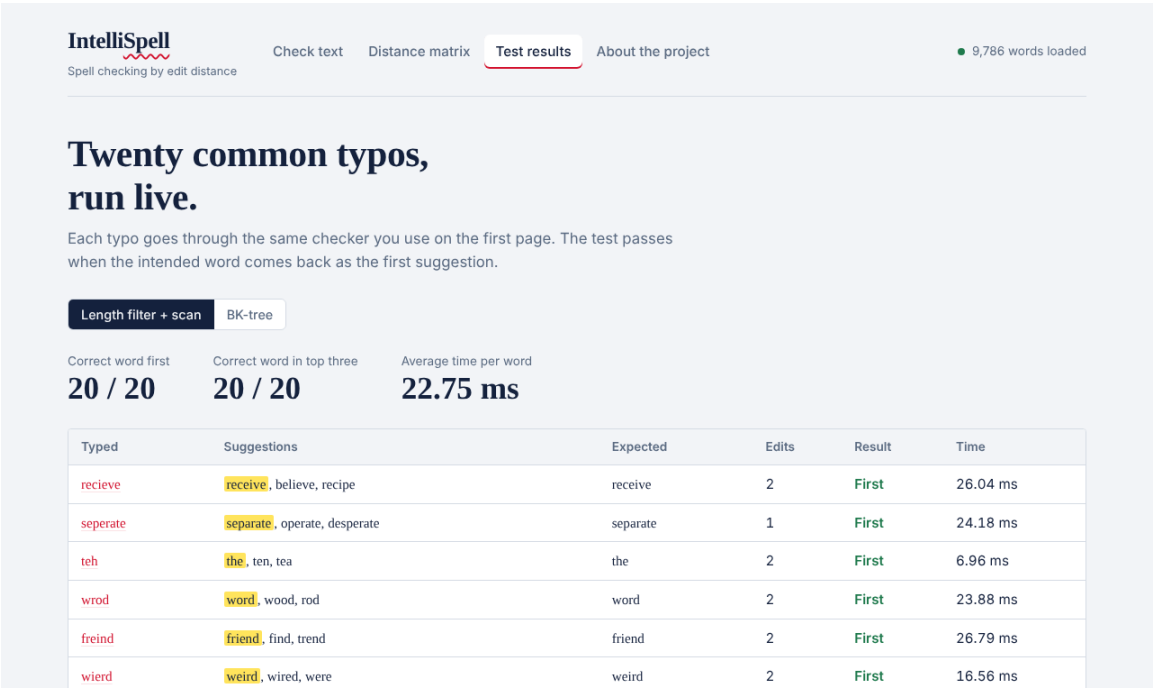


Fig. 3

8. CONCLUSION & FUTURE SCOPE

IntelliSpell meets its objectives. A classic algorithm, applied well, gives precise, private and explainable spell checking with no server required.

- **Accurate:** edit distance finds the closest real words; 20 of 20 test typos ranked correctly first.
- **Fast:** a hash-set lookup and a length filter keep checks near-instant.
- **Private:** text and files are processed in the browser and never uploaded.
- **Transparent:** the matrix view shows exactly how each distance is worked out.

Future Scope: the following are proposed enhancements, not features of the current build.

- **Full Damerau-Levenshtein:** count swaps as one edit everywhere, not only in ranking.
- **Bigger, multilingual dictionaries:** plus a saved personal dictionary.
- **Context-aware checking:** catch real-word errors such as there vs their.
- **Faster search:** BK-tree or trie instead of scanning by length.
- **OCR for scanned PDFs:** read image-only documents.
- **Keep original format:** export corrected .docx or .pdf, not only .txt.

9. REFERENCES

1. F. J. Damerau, “A technique for computer detection and correction of spelling errors,” Communications of the ACM, vol. 7, no. 3, pp. 171–176, 1964.

2. V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

3. W. A. Burkhard and R. M. Keller, “Some approaches to best-match file searching,” Communications of the ACM, vol. 16, no. 4, pp. 230–236, 1973.

4. R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” Journal of the ACM, vol. 21, no. 1, pp. 168–173, 1974..

5. P. Norvig, “How to Write a Spelling Corrector,” 2007. [Online]. Available: https://norvig.com/spell-correct.html

6. Mozilla, “PDF.js,” [Online]. Available: https://mozilla.github.io/pdf.js/

7. mammoth.js, “Convert Word documents (.docx) to HTML,” [Online]. Available: https://github.com/mwilliamson/mammoth.js